

LOGICAL STRUCTURES

Lecture 7. Undecidability of predicate logic and Tableau Algorithm

Advanced Program in Computer Science

Spring, 2025

Contents

- ① Undecidability of predicate logic
- ② **Tableau algorithm:**
 - ① for propositional logic
 - ② for predicate logic

Undecidability of predicate logic

- in propositional logic, in principle, we can determine whether $\models \phi$ holds by constructing truth table

Undecidability of predicate logic

- in propositional logic, in principle, we can determine whether $\models \phi$ holds by constructing truth table
- for predicate logic, that mechanical procedure does not work

Undecidability of predicate logic

- in propositional logic, in principle, we can determine whether $\models \phi$ holds by constructing truth table
- for predicate logic, that mechanical procedure does not work
- the problem of determining whether a predicate logic formula is valid is known as a *decision problem*.

Example 1.1

Show that the following sequents do not hold

- a $\forall x(P(x) \rightarrow R(x)), \forall x(Q(x) \rightarrow R(x)) \vdash \exists x(P(x) \wedge Q(x))$
- b $\forall x \exists y S(x, y) \vdash \exists y \forall x S(x, y)$
- c $\exists x(\neg P(x) \wedge Q(x)) \vdash \forall x(P(x) \rightarrow Q(x))$

Decision problem

Definition 1.2

A *solution* to a decision problem is a program (written in Java, C, or any common language) that takes the problem instances as input and **always** terminates, producing a correct **yes** or **no** output.

Decision problem

Definition 1.2

A *solution* to a decision problem is a program (written in Java, C, or any common language) that takes the problem instances as input and **always** terminates, producing a correct **yes** or **no** output.

- For predicate logic, the input is an arbitrary formula ϕ of predicate logic and the program is correct if it produces **yes** whenever the input formula is valid and **no** whenever it is not.

Decision problem

Definition 1.2

A **solution** to a decision problem is a program (written in Java, C, or any common language) that takes the problem instances as input and **always** terminates, producing a correct **yes** or **no** output.

- For predicate logic, the input is an arbitrary formula ϕ of predicate logic and the program is correct if it produces **yes** whenever the input formula is valid and **no** whenever it is not.

Decision problem: Validity in predicate logic

Given a logical formula ϕ in predicate logic, does $\vdash \phi$ hold, **yes** or **no**?

Decision problem

- **Guess:** not solvable.

Decision problem

- **Guess:** not solvable.
- **Technique:** take some other problem of which we know that is not solvable, and show that the solvability of our problem entails the solvability of the other one.

Decision problem

- **Guess:** not solvable.
- **Technique:** take some other problem of which we know that is not solvable, and show that the solvability of our problem entails the solvability of the other one.

The Post correspondence problem

Given a finite sequence of pairs $(s_1, t_1), (s_2, t_2), \dots, (s_k, t_k)$ such that all s_i and t_i are binary strings of positive length, is there a sequence of indices $i_1, i_2, \dots, i_n$ with $n \geq 1$ such that the concatenation of strings $s_{i_1} s_{i_2} \dots s_{i_n}$ equals $t_{i_1} t_{i_2} \dots t_{i_n}$.

Example 1.3

Instance 1: $C = ((1, 101), (10, 00), (011, 11))$.

$$\begin{array}{lll} s_1 \stackrel{\text{def}}{=} 1 & s_2 \stackrel{\text{def}}{=} 10 & s_3 \stackrel{\text{def}}{=} 011 \\ t_1 \stackrel{\text{def}}{=} 101 & t_2 \stackrel{\text{def}}{=} 00 & t_3 \stackrel{\text{def}}{=} 11. \end{array}$$

Example 1.3

Instance 1: $C = ((1, 101), (10, 00), (011, 11))$.

$$\begin{array}{lll} s_1 \stackrel{\text{def}}{=} 1 & s_2 \stackrel{\text{def}}{=} 10 & s_3 \stackrel{\text{def}}{=} 011 \\ t_1 \stackrel{\text{def}}{=} 101 & t_2 \stackrel{\text{def}}{=} 00 & t_3 \stackrel{\text{def}}{=} 11. \end{array}$$

A solution: consider the sequence of indices $(1, 3, 2, 3)$ then both $s_1 s_3 s_2 s_3$ and $t_1 t_3 t_2 t_3$ equal 101110011.

Example 1.4

Instance 2: $C = ((001, 0), (01, 011), (01, 101), (10, 001))$.

$$s_2 \stackrel{\text{def}}{=} 01$$

$$s_3 \stackrel{\text{def}}{=} 01$$

$$s_4 \stackrel{\text{def}}{=} 10$$

$$t_2 \stackrel{\text{def}}{=} 011$$

$$t_3 \stackrel{\text{def}}{=} 101$$

$$t_4 \stackrel{\text{def}}{=} 001$$

Example 1.4

Instance 2: $C = ((001, 0), (01, 011), (01, 101), (10, 001))$.

$$\begin{array}{lll} s_2 \stackrel{\text{def}}{=} 01 & s_3 \stackrel{\text{def}}{=} 01 & s_4 \stackrel{\text{def}}{=} 10 \\ t_2 \stackrel{\text{def}}{=} 011 & t_3 \stackrel{\text{def}}{=} 101 & t_4 \stackrel{\text{def}}{=} 001 \end{array}$$

Any solution?

- hand picking or computer program?
- note that the same number can occur in the sequence of indices, so **no exhaustive search**.

Example 1.4

Instance 2: $C = ((001, 0), (01, 011), (01, 101), (10, 001))$.

$$\begin{array}{lll} s_2 \stackrel{\text{def}}{=} 01 & s_3 \stackrel{\text{def}}{=} 01 & s_4 \stackrel{\text{def}}{=} 10 \\ t_2 \stackrel{\text{def}}{=} 011 & t_3 \stackrel{\text{def}}{=} 101 & t_4 \stackrel{\text{def}}{=} 001 \end{array}$$

Any solution?

- hand picking or computer program?
- note that the same number can occur in the sequence of indices, so **no exhaustive search**.

Bad news: The Post correspondence problem is not solvable.

Decision problem

Theorem 1.5

The decision problem of validity in predicate logic is undecidable: no program exists which, given ϕ , decides whether $\models \phi$.

Proof. Textbook, page 133-135.

Tableau algorithm for propositional logic

Suppose that C is a list of formulas, then C can be extended using the following *tableau expansion rules*:

Tableau algorithm for propositional logic

Suppose that C is a list of formulas, then C can be extended using the following *tableau expansion rules*:

- *$\wedge$ -rule*: if $\phi \wedge \psi \in C$, then $C := C \cup \{\phi, \psi\}$.

Tableau algorithm for propositional logic

Suppose that C is a list of formulas, then C can be extended using the following *tableau expansion rules*:

- *$\wedge$ -rule*: if $\phi \wedge \psi \in C$, then $C := C \cup \{\phi, \psi\}$.
- *$\neg\neg$ -rule*: if $\neg\neg\phi \in C$, then $C := C \cup \{\phi\}$.

Tableau algorithm for propositional logic

Suppose that C is a list of formulas, then C can be extended using the following *tableau expansion rules*:

- **$\wedge$ -rule:** if $\phi \wedge \psi \in C$, then $C := C \cup \{\phi, \psi\}$.
- **$\neg\neg$ -rule:** if $\neg\neg\phi \in C$, then $C := C \cup \{\phi\}$.
- **$\neg\vee$ -rule:** if $\neg(\phi \vee \psi) \in C$, then $C := C \cup \{\neg\phi, \neg\psi\}$.

Tableau algorithm for propositional logic

Suppose that C is a list of formulas, then C can be extended using the following *tableau expansion rules*:

- **$\wedge$ -rule**: if $\phi \wedge \psi \in C$, then $C := C \cup \{\phi, \psi\}$.
- **$\neg\neg$ -rule**: if $\neg\neg\phi \in C$, then $C := C \cup \{\phi\}$.
- **$\neg\vee$ -rule**: if $\neg(\phi \vee \psi) \in C$, then $C := C \cup \{\neg\phi, \neg\psi\}$.
- **$\vee$ -rule**: if $\phi \vee \psi \in C$, then $C := C \cup \{\phi\}$ **or** $C := C \cup \{\psi\}$.

Tableau algorithm for propositional logic

Suppose that C is a list of formulas, then C can be extended using the following *tableau expansion rules*:

- **$\wedge$ -rule**: if $\phi \wedge \psi \in C$, then $C := C \cup \{\phi, \psi\}$.
- **$\neg\neg$ -rule**: if $\neg\neg\phi \in C$, then $C := C \cup \{\phi\}$.
- **$\neg\vee$ -rule**: if $\neg(\phi \vee \psi) \in C$, then $C := C \cup \{\neg\phi, \neg\psi\}$.
- **$\vee$ -rule**: if $\phi \vee \psi \in C$, then $C := C \cup \{\phi\}$ **or** $C := C \cup \{\psi\}$.
- **$\neg\wedge$ -rule**: if $\neg(\phi \wedge \psi) \in C$, then $C := C \cup \{\neg\phi\}$ **or** $C := C \cup \{\neg\psi\}$.

Example 2.1

Apply Tableau Expansion Rules to $\neg(\neg p \wedge q) \wedge \neg p$.

Example 2.1

Apply Tableau Expansion Rules to $\neg(\neg p \wedge q) \wedge \neg p$.

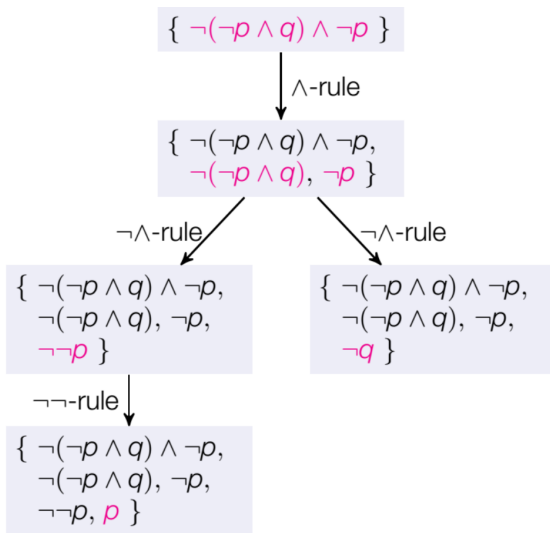


Tableau paths

Definition 2.2

A *tableau path* is a sequence of set of formulas $C_0, C_1, \dots, C_n$ such that for all $i \in \{1, 2, \dots, n\}$:

Tableau paths

Definition 2.2

A *tableau path* is a sequence of set of formulas $C_0, C_1, \dots, C_n$ such that for all $i \in \{1, 2, \dots, n\}$:

- C_i does not contains any *clash*, i.e, $\{p, \neg p\}$

Tableau paths

Definition 2.2

A **tableau path** is a sequence of set of formulas $C_0, C_1, \dots, C_n$ such that for all $i \in \{1, 2, \dots, n\}$:

- C_i does not contains any **clash**, i.e, $\{p, \neg p\}$
- C_i is the result of applying a Tableau Expansion Rule to C_j ($j < i$).

Tableau paths

Definition 2.2

A **tableau path** is a sequence of set of formulas $C_0, C_1, \dots, C_n$ such that for all $i \in \{1, 2, \dots, n\}$:

- C_i does not contains any **clash**, i.e, $\{p, \neg p\}$
- C_i is the result of applying a Tableau Expansion Rule to C_j ($j < i$).

Such a path is:

- **complete** if no expansion rule can be applied to C_n ;

Tableau paths

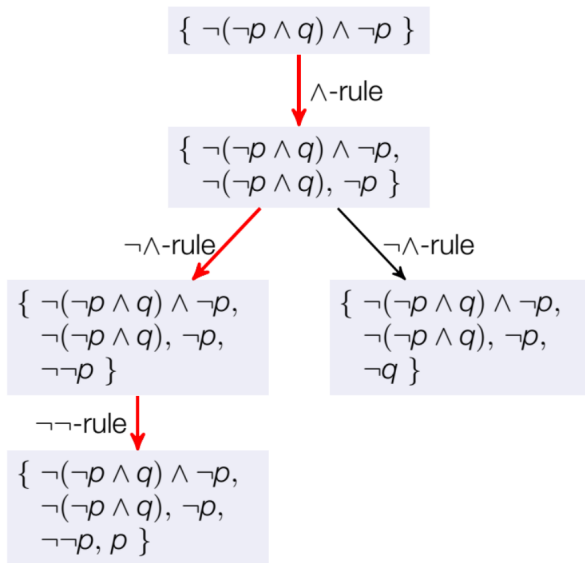
Definition 2.2

A **tableau path** is a sequence of set of formulas $C_0, C_1, \dots, C_n$ such that for all $i \in \{1, 2, \dots, n\}$:

- C_i does not contains any **clash**, i.e, $\{p, \neg p\}$
- C_i is the result of applying a Tableau Expansion Rule to C_j ($j < i$).

Such a path is:

- **complete** if no expansion rule can be applied to C_n ;
- **closed** if C_n contains a clash $\{p, \neg p\}$.



complete & closed

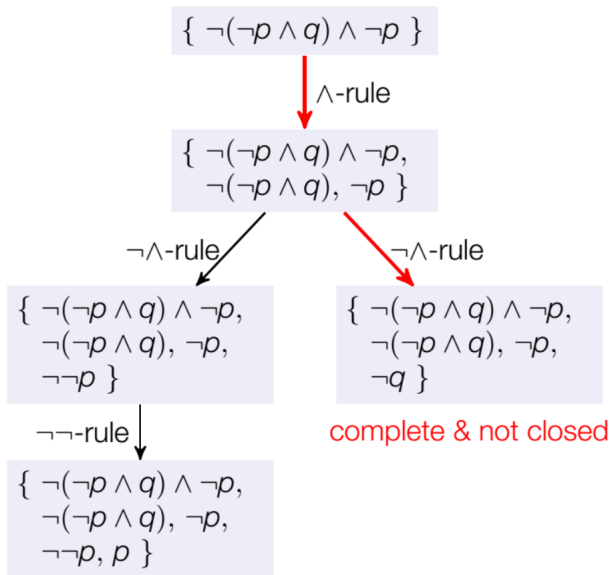


Tableau Algorithm

Input: A propositional formula ϕ .

Output: **satisfiable** if ϕ is satisfiable; otherwise **unsatisfiable**.

Tableau Algorithm

Input: A propositional formula ϕ .

Output: **satisfiable** if ϕ is satisfiable; otherwise **unsatisfiable**.

- 1 Generate tableau paths starting with $C_0 := \{\phi\}$

Tableau Algorithm

Input: A propositional formula ϕ .

Output: **satisfiable** if ϕ is satisfiable; otherwise **unsatisfiable**.

- 1 Generate tableau paths starting with $C_0 := \{\phi\}$
- 2 If there is a complete tableau path that is not closed, output **satisfiable**.

Tableau Algorithm

Input: A propositional formula ϕ .

Output: **satisfiable** if ϕ is satisfiable; otherwise **unsatisfiable**.

- 1 Generate tableau paths starting with $C_0 := \{\phi\}$
- 2 If there is a complete tableau path that is not closed, output **satisfiable**.
- 3 If all complete tableau paths are closed, output **unsatisfiable**.

Tableau Algorithm

Input: A propositional formula ϕ .

Output: **satisfiable** if ϕ is satisfiable; otherwise **unsatisfiable**.

- 1 Generate tableau paths starting with $C_0 := \{\phi\}$
- 2 If there is a complete tableau path that is not closed, output **satisfiable**.
- 3 If all complete tableau paths are closed, output **unsatisfiable**.

Examples

Verify if the following propositional formulas are satisfiable

❶ $\phi = (\neg p \wedge q) \wedge \neg \neg r$

❷ $\phi = (p \wedge q) \wedge \neg(p \vee q)$

❸ $\phi = ((p \rightarrow q) \wedge p) \wedge \neg \neg q$

❹ $\phi = ((\neg p \vee q) \wedge p) \wedge \neg q$

Examples

Verify if the following propositional formulas are satisfiable

❶ $\phi = (\neg p \wedge q) \wedge \neg \neg r$

❷ $\phi = (p \wedge q) \wedge \neg(p \vee q)$

❸ $\phi = ((p \rightarrow q) \wedge p) \wedge \neg \neg q$

❹ $\phi = ((\neg p \vee q) \wedge p) \wedge \neg q$

Answers:

- 1,3: satisfiable
- 2,4: unsatisfiable.

Tableau Method for predicate logic

Example 2.3

Is $\forall x \forall y R(x, y) \wedge \neg R(c, c)$ satisfiable?

Tableau Method for predicate logic

Example 2.3

Is $\forall x \forall y R(x, y) \wedge \neg R(c, c)$ satisfiable?

$\Rightarrow$ **Extended tableau method**

Extended Tableau Expansion Rules

Suppose that C is a list of predicate formulas, then C can be extended using the following *tableau expansion rules*:

Extended Tableau Expansion Rules

Suppose that C is a list of predicate formulas, then C can be extended using the following *tableau expansion rules*:

- 1 *$\wedge$ -rule*: if $\phi \wedge \psi \in C$, then $C := C \cup \{\phi, \psi\}$.

Extended Tableau Expansion Rules

Suppose that C is a list of predicate formulas, then C can be extended using the following *tableau expansion rules*:

- ① *$\wedge$ -rule*: if $\phi \wedge \psi \in C$, then $C := C \cup \{\phi, \psi\}$.
- ② *$\neg\neg$ -rule*: if $\neg\neg\phi \in C$, then $C := C \cup \{\phi\}$.

Extended Tableau Expansion Rules

Suppose that C is a list of predicate formulas, then C can be extended using the following *tableau expansion rules*:

- ❶ **$\wedge$ -rule**: if $\phi \wedge \psi \in C$, then $C := C \cup \{\phi, \psi\}$.
- ❷ **$\neg\neg$ -rule**: if $\neg\neg\phi \in C$, then $C := C \cup \{\phi\}$.
- ❸ **$\neg\vee$ -rule**: if $\neg(\phi \vee \psi) \in C$, then $C := C \cup \{\neg\phi, \neg\psi\}$.

Extended Tableau Expansion Rules

Suppose that C is a list of predicate formulas, then C can be extended using the following *tableau expansion rules*:

- ❶ **$\wedge$ -rule**: if $\phi \wedge \psi \in C$, then $C := C \cup \{\phi, \psi\}$.
- ❷ **$\neg\neg$ -rule**: if $\neg\neg\phi \in C$, then $C := C \cup \{\phi\}$.
- ❸ **$\neg\vee$ -rule**: if $\neg(\phi \vee \psi) \in C$, then $C := C \cup \{\neg\phi, \neg\psi\}$.
- ❹ **$\vee$ -rule**: if $\phi \vee \psi \in C$, then $C := C \cup \{\phi\}$ **or** $C := C \cup \{\psi\}$.

Extended Tableau Expansion Rules

Suppose that C is a list of predicate formulas, then C can be extended using the following *tableau expansion rules*:

- ❶ **$\wedge$ -rule**: if $\phi \wedge \psi \in C$, then $C := C \cup \{\phi, \psi\}$.
- ❷ **$\neg\neg$ -rule**: if $\neg\neg\phi \in C$, then $C := C \cup \{\phi\}$.
- ❸ **$\neg\vee$ -rule**: if $\neg(\phi \vee \psi) \in C$, then $C := C \cup \{\neg\phi, \neg\psi\}$.
- ❹ **$\vee$ -rule**: if $\phi \vee \psi \in C$, then $C := C \cup \{\phi\}$ **or** $C := C \cup \{\psi\}$.
- ❺ **$\neg\wedge$ -rule**: if $\neg(\phi \wedge \psi) \in C$, then $C := C \cup \{\neg\phi\}$ **or** $C := C \cup \{\neg\psi\}$.

- ⑥ **$\exists$ -rule:** if $\exists x\phi \in C$ and $\phi[c/x] \notin C$ for all constant symbols c , then $C := C \cup \{\phi[c/x]\}$ where c does not occur in C .

- ⑥ **$\exists$ -rule:** if $\exists x\phi \in C$ and $\phi[c/x] \notin C$ for all constant symbols c , then $C := C \cup \{\phi[c/x]\}$ where c does not occur in C .
- ⑦ **$\forall$ -rule:** if $\forall x\phi \in C$ and $\phi[c/x] \notin C$ for some constant symbols c , then $C := C \cup \{\phi[c/x]\}$ with $\phi[c/x] \notin C$.

- ⑥ **$\exists$ -rule:** if $\exists x\phi \in C$ and $\phi[c/x] \notin C$ for all constant symbols c , then $C := C \cup \{\phi[c/x]\}$ where c does not occur in C .
- ⑦ **$\forall$ -rule:** if $\forall x\phi \in C$ and $\phi[c/x] \notin C$ for some constant symbols c , then $C := C \cup \{\phi[c/x]\}$ with $\phi[c/x] \notin C$.
If c does not occur in C , then the application of the $\forall$ -rule is said to **create a new constant**.

- ⑥ **$\exists$ -rule:** if $\exists x\phi \in C$ and $\phi[c/x] \notin C$ for all constant symbols c , then $C := C \cup \{\phi[c/x]\}$ where c does not occur in C .
- ⑦ **$\forall$ -rule:** if $\forall x\phi \in C$ and $\phi[c/x] \notin C$ for some constant symbols c , then $C := C \cup \{\phi[c/x]\}$ with $\phi[c/x] \notin C$.
If c does not occur in C , then the application of the $\forall$ -rule is said to **create a new constant**.
- ⑧ **$\neg\exists$ -rule:** if $\neg\exists x\phi \in C$, then $C := C \cup \{\forall x\neg\phi\}$

- ⑥ **$\exists$ -rule:** if $\exists x\phi \in C$ and $\phi[c/x] \notin C$ for all constant symbols c , then $C := C \cup \{\phi[c/x]\}$ where c does not occur in C .
- ⑦ **$\forall$ -rule:** if $\forall x\phi \in C$ and $\phi[c/x] \notin C$ for some constant symbols c , then $C := C \cup \{\phi[c/x]\}$ with $\phi[c/x] \notin C$.
If c does not occur in C , then the application of the $\forall$ -rule is said to **create a new constant**.
- ⑧ **$\neg\exists$ -rule:** if $\neg\exists x\phi \in C$, then $C := C \cup \{\forall x\neg\phi\}$
- ⑨ **$\neg\forall$ -rule:** if $\neg\forall x\phi \in C$, then $C := C \cup \{\exists x\neg\phi\}$

Tableau paths

Definition 2.4

A *tableau path* is a sequence of set of formulas $C_0, C_1, \dots$ such that for all $i \geq 1$:

Tableau paths

Definition 2.4

A **tableau path** is a sequence of set of formulas $C_0, C_1, \dots$ such that for all $i \geq 1$:

- C_i does not contains any **clash**, i.e, $\{t, \neg t\}$, for any term t ;

Tableau paths

Definition 2.4

A **tableau path** is a sequence of set of formulas $C_0, C_1, \dots$ such that for all $i \geq 1$:

- C_i does not contains any **clash**, i.e, $\{t, \neg t\}$, for any term t ;
- C_i is the result of applying a Tableau Expansion Rule to C_j ($j < i$)).

Tableau paths

Definition 2.4

A **tableau path** is a sequence of set of formulas $C_0, C_1, \dots$ such that for all $i \geq 1$:

- C_i does not contains any **clash**, i.e, $\{t, \neg t\}$, for any term t ;
- C_i is the result of applying a Tableau Expansion Rule to C_j ($j < i$)).

Such a path is:

- **complete** if no expansion rule can be applied to C_n ;

Tableau paths

Definition 2.4

A **tableau path** is a sequence of set of formulas $C_0, C_1, \dots$ such that for all $i \geq 1$:

- C_i does not contains any **clash**, i.e, $\{t, \neg t\}$, for any term t ;
- C_i is the result of applying a Tableau Expansion Rule to C_j ($j < i$)).

Such a path is:

- **complete** if no expansion rule can be applied to C_n ;
- **containing a clash** if C_n contains a clash $\{t, \neg t\}$.

Tableau paths

Definition 2.4

A **tableau path** is a sequence of set of formulas $C_0, C_1, \dots$ such that for all $i \geq 1$:

- C_i does not contains any **clash**, i.e, $\{t, \neg t\}$, for any term t ;
- C_i is the result of applying a Tableau Expansion Rule to C_j ($j < i$)).

Such a path is:

- **complete** if no expansion rule can be applied to C_n ;
- **containing a clash** if C_n contains a clash $\{t, \neg t\}$.

Note: Every tableau path containing a clash is also complete.

$$\mathcal{C}_0 \quad \{ \forall x \forall y R(x, y) \wedge \neg R(c, c) \}$$

$\wedge$ -rule

$$\mathcal{C}_1 \quad \{ \forall x \forall y R(x, y) \wedge \neg R(c, c), \\ \forall x \forall y R(x, y), \neg R(c, c) \}$$

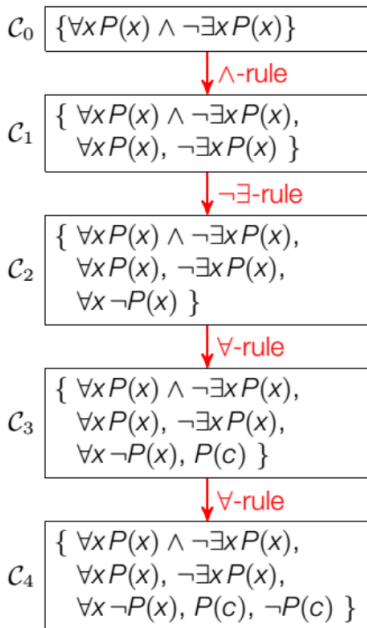
$\forall$ -rule

$$\mathcal{C}_2 \quad \{ \forall x \forall y R(x, y) \wedge \neg R(c, c), \\ \forall x \forall y R(x, y), \neg R(c, c), \\ \forall y R(c, y) \}$$

$\forall$ -rule

$$\mathcal{C}_3 \quad \{ \forall x \forall y R(x, y) \wedge \neg R(c, c), \\ \forall x \forall y R(x, y), \neg R(c, c), \\ \forall y R(c, y), R(c, c) \}$$

contains a clash
is complete



contains a clash

- the first application of $\forall$ -rule creates a new constant
- the second one does not.

Fairness

Definition 2.5

A tableau path $C_0, C_1, \dots$ is *fair* if $\forall$ -rules that create new constants are only applied if no other rule can be applied.

Tableau method

Theorem 2.6

*A formula ϕ in predicate logic is **unsatisfiable** iff every complete fair tableau path starting with $\{\phi\}$ contains a clash.*

Tableau method

Theorem 2.6

*A formula ϕ in predicate logic is **unsatisfiable** iff every complete fair tableau path starting with $\{\phi\}$ contains a clash.*

Recall: ϕ is a tautology $\Leftrightarrow \neg\phi$ is unsatisfiable.

Tableau method

Theorem 2.6

*A formula ϕ in predicate logic is **unsatisfiable** iff every complete fair tableau path starting with $\{\phi\}$ contains a clash.*

Recall: ϕ is a tautology $\Leftrightarrow \neg\phi$ is unsatisfiable.

Corollary: a formula ϕ is valid (tautology) if and only if every complete fair tableau path starting with $\{\neg\phi\}$ contains a clash.

$$\mathcal{C}_0 \quad \{ \forall x \forall y R(x, y) \wedge \neg R(c, c) \}$$

$\wedge$ -rule

$$\mathcal{C}_1 \quad \{ \forall x \forall y R(x, y) \wedge \neg R(c, c), \\ \forall x \forall y R(x, y), \neg R(c, c) \}$$

$\forall$ -rule

$$\mathcal{C}_2 \quad \{ \forall x \forall y R(x, y) \wedge \neg R(c, c), \\ \forall x \forall y R(x, y), \neg R(c, c), \\ \forall y R(c, y) \}$$

unsatisfiable

$\forall$ -rule

$$\mathcal{C}_3 \quad \{ \forall x \forall y R(x, y) \wedge \neg R(c, c), \\ \forall x \forall y R(x, y), \neg R(c, c), \\ \forall y R(c, y), R(c, c) \}$$

contains a clash
is complete

$$C_0 \quad \{ \forall x P(x) \wedge \neg \exists x P(x) \}$$

↓ $\wedge$ -rule

$$C_1 \quad \{ \forall x P(x) \wedge \neg \exists x P(x), \\ \forall x P(x), \neg \exists x P(x) \}$$

↓ $\neg \exists$ -rule

$$C_2 \quad \{ \forall x P(x) \wedge \neg \exists x P(x), \\ \forall x P(x), \neg \exists x P(x), \\ \forall x \neg P(x) \}$$

↓ $\forall$ -rule

$$C_3 \quad \{ \forall x P(x) \wedge \neg \exists x P(x), \\ \forall x P(x), \neg \exists x P(x), \\ \forall x \neg P(x), P(c) \}$$

↓ $\forall$ -rule

$$C_4 \quad \{ \forall x P(x) \wedge \neg \exists x P(x), \\ \forall x P(x), \neg \exists x P(x), \\ \forall x \neg P(x), P(c), \neg P(c) \}$$

unsatisfiable

contains a clash

Examples

- ❶ $\exists x P(x) \wedge \forall x \neg(P(x) \vee Q(x))$ is unsatisfiable.
- ❷ $\exists x \exists y R(x, y) \wedge \neg R(c, d)$ is satisfiable.